

A PROPOSED ABI FOR AN ASYNC-FIRST KERNEL

2026-07-12

Contents

1	Introduction.....	5
1.1	Why this document exists	5
1.1.1	sitas	5
1.1.2	CharlotteOS	5
1.2	Who this is for	5
1.3	The thesis in one sentence.....	5
1.4	Document structure.....	6
2	Architecture.....	7
2.1	The logical processor.....	7
2.2	Per-LP state	7
2.3	The observer / waker model.....	7
2.3.1	Observable	7
2.3.2	Observer	8
2.3.3	Wiring	8
2.4	The scheduler.....	8
2.5	Inter-processor interrupts (IPIs)	8
2.6	Capability tables.....	9
3	Execution model.....	10
3.1	Thread lifecycle.....	10
3.1.1	Entry	10
3.1.2	Blocking	10
3.1.3	Waking.....	10
3.1.4	Yielding.....	10

3.2	The quantum timer	10
3.3	The block-yield-wake cycle.....	11
3.4	Context saving	11
4	The async syscall ABI	12
4.1	The five operations.....	12
4.1.1	submit — start an async operation	12
4.1.2	wait — the reactor’s only sleep	12
4.1.3	poll — non-blocking check	12
4.1.4	cancel — drop-as-cancellation.....	12
4.1.5	close — release a slot	12
4.2	The exit-observer completion path	13
4.3	The CQ ring (completion queue).....	13
4.4	Cancellation and the buffer-reclaim contract.....	13
4.5	Backpressure	13
5	Programming model	15
5.1	Invariants.....	15
5.1.1	Only the owning LP mutates its state	15
5.1.2	Messages are owned values	15
5.1.3	References to shard-local state must not escape	15
5.1.4	Work stays on its assigned LP	15
5.1.5	Observability returns owned snapshots	15
5.2	Primitives.....	16
5.2.1	ShardLocal<T>	16
5.2.2	ShardMailbox<M>	16
5.2.3	IPI closure.....	16
5.2.4	Completion API.....	16
5.2.5	CQ ring	16

5.3	Rule-of-thumb guidance	16
6	Development	17
6.1	Build targets	17
6.2	Self-tests	17
6.3	Async-syscall demo	17
6.4	CI.....	18
6.5	Where to start reading	18
7	Current status.....	19
7.1	Verified (boots on QEMU, zero panics).....	19
7.2	Known limitations	19
7.3	Prior art / independent verification	19
8	The Collaboration: A Retrospective	20
8.1	The convergence	20
8.2	The structural correspondence.....	20
8.3	The exploration phases ($A \rightarrow C \rightarrow B$)	20
8.3.1	Phase A — OS backend seam (sitas)	21
8.3.2	Phase C — Async syscall ABI (CharlotteOS kernel)	21
8.3.3	Phase B — sitas's discipline inside the kernel	22
8.3.4	Beyond the plan — what was built additionally	22
8.4	The async syscall ABI in detail.....	23
8.4.1	The five operations	23
8.4.2	The exit-observer completion path	23
8.4.3	The CQ ring	23
8.4.4	Cancellation and the buffer-reclaim contract.....	24
8.4.5	Backpressure	24

8.5	The bugs we found (and the real value of running on hardware)	24
8.5.1	Panic handler recursed through heap-dependent <code>logln</code>	24
8.5.2	Blocked threads lost their execution context	24
8.5.3	RwLock held-across-call deadlocks in <code>sleep()</code> and <code>abort()</code>	25
8.5.4	Quantum timer grew the timer queue without bound	25
8.5.5	A thread cannot free its own kernel stack	25
8.5.6	Stack allocator never registered guard pages	25
8.5.7	LA57 paging-mode mismatch on x86-64	26
8.5.8	ASID-0 collision (“the TLB bug that wasn’t”)	26
8.5.9	x86-64 <code>kernel_thread_trampoline</code> halted on thread return	26
8.6	Verification	26
8.6.1	Self-test suite (both architectures pass)	27
8.6.2	Async-syscall demo (both architectures)	27
8.6.3	sitas integration	27
8.6.4	CI	27
8.7	What we learned	27
8.8	Current status (July 2026)	28
8.9	Repositories and branches	29
9	Glossary	30

1 Introduction

1.1 Why this document exists

Two independent experiments arrived at the same shape from opposite directions.

1.1.1 *sitas*

sitas starts in userspace and asks: *what does shared-nothing, shard-per-core concurrency look like when it is built out of Rust ownership instead of locks?* Its answer is a discipline: each shard owns its state, only the owning shard mutates it, and everything that crosses a shard boundary is an owned value moved through an explicit typed message. On top of that it grows a custom async executor, per-shard reactors, `io_uring` completion I/O, and snapshot-based observability.

1.1.2 CharlotteOS

CharlotteOS starts in the kernel and asks: *hardware is asynchronous with respect to the CPU, so why is the OS synchronous?* Its answer is to make the kernel async-first — cheap threads, an observer/waker event model, and system calls that are asynchronous by default, returning a completion capability that userspace can wait on.

Both projects converged on the same primitive: a waitable completion handle. This document is about what happens when they meet: a userspace runtime whose async model **agrees** with the kernel it runs on, instead of emulating async-first on a blocking foundation. That agreement is the whole point — it removes the “retrofit tax” (async-signal-safety, thread pools, `io_uring`-over-blocking-AIO) that exists only because the runtime and the OS disagree about what is fundamental.

1.2 Who this is for

This document is written for developers. It assumes familiarity with Rust (ownership, `Send`, `Future`), concurrency primitives (threads, atomics, message passing), and basic OS architecture (scheduling, interrupts, address spaces).

Every claim is based on what has actually been built and verified (on QEMU AArch64 and x86-64). Unimplemented ideas are flagged as such.

1.3 The thesis in one sentence

Hardware is asynchronous; the OS and the userspace runtime should be too. *CharlotteOS* and *sitas* both *start* from that premise instead of retrofitting async onto a synchronous base. They are two halves of the same idea, originally built independently, now converging.

1.4 Document structure

- **Chapter 2** — Architecture: the kernel’s structural model (Logical Processors, Observers, per-LP state, the scheduler).
- **Chapter 3** — Execution model: threads, blocking, context switching, timers, and the block-wake path.
- **Chapter 4** — The async syscall ABI: completions, submission, the CQ ring, cancellation, and back-pressure.
- **Chapter 5** — Programming model: invariants, ShardLocal, ShardMailbox, and rule-of-thumb guidance for kernel code.
- **Chapter 6** — Development: build targets, QEMU boot, the self-test framework, and CI.
- **Chapter 7** — Current status: what exists, what is verified, what is known-broken or deferred.

2 Architecture

2.1 The logical processor

A **Logical Processor** (LP) is one hardware core. The kernel shards its state per-LP using `PerLp<T>`: an array of `RwLock<T>`, one slot per LP, indexed by the LP's identity. The LP identity is stored in a register (`TPIDR_EL1` on AArch64, `TSC_AUX` on x86-64) and read with a single instruction — it is always safe to call from any context, including interrupt handlers.

LP placement is **not advisory**: an LP *is* a core. When a sitas shard is bound to an LP it is bound to a core by construction. No `sched_setaffinity` race, no `cpuset` surprises.

2.2 Per-LP state

2.2.0.0.1 `PerLp<T>`

A boxed slice of `RwLock<T>`, one per LP. The lock masks interrupts on acquire, preventing ISR self-deadlock. Used for data accessed from both thread context and interrupt context (timer queues, LAPIC registers, interrupt depth counters).

2.2.0.0.2 `ShardLocal<T>`

(newer, sitas-inspired.) A lock-free per-LP container that stores its slots behind `UnsafeCell<T>` rather than `RwLock<T>`. Access is through a closure (`with(fn)` or `try_with(fn)`) that asserts:

- The caller is on the owning LP (panics otherwise).
- The slot is not already borrowed by a re-entrant call (a per-LP `AtomicBool` borrow flag, RAIL-guarded).

The closure ensures the `&mut T` never escapes the accessor. Use `PerLp<T>` when interrupt-safety or cross-LP access is needed; use `ShardLocal<T>` for single-LP, thread-only data (it has no spin-lock acquire cost).

2.3 The observer / waker model

CharlotteOS's event mechanism reduces to one pattern: a thread registers its `Waker` (an `Observer` wrapping its `ThreadId`) on an `Observable`. When the observable fires, it calls `Observer::notify()`, which submits the thread to the system scheduler as ready.

2.3.1 Observable

```
trait fn register_observer(&self, observer: Weak<dyn Observer>).
```


2.3.2 Observer

`trait: fn notify(self: Arc<Self>)` — called by an `Observable` when the event occurs. The kernel's `Waker` implements `Observer` by calling `submit_ready_thread(tid)`, queueing the thread on the least-loaded LP.

2.3.3 Wiring

`sleep(50ms)` creates a `TimerEvent` (`Observable`), calls `block_thread(tid, &event)` to register the thread's `Waker` on the event and mark the thread `Blocked`, pushes the event into the per-LP timer queue, and yields. When the timer IRQ fires it processes the event, calls `signal()` which wakes the thread. The completion-capability model uses the exact same mechanism: a `Completion` is `Observable`, and `wait(cap)` blocks on it.

2.4 The scheduler

The system scheduler (`SYSTEM_SCHEDULER`) is a per-LP collection of pluggable `LpScheduler` instances. The only current implementation is `RoundRobin`: threads are queued in a FIFO, and a per-LP quantum timer (10 ms) marks a context-switch-pending flag.

Key operations:

- `spawn_thread(asid, entry)` — creates a `Thread`, adds it to the master table, submits it to the scheduler. Returns a `ThreadId`.
- `block_thread(tid, &event)` — registers the thread's `Waker` on the observable, marks the thread `Blocked`, removes it from the run queue. The thread saves its context on the subsequent `yield_lp()` and resumes from the same point when woken.
- `submit_ready_thread(tid)` — re-admits a thread to the least-loaded LP's run queue. Used by `Waker::notify()` and other wake paths.
- `yield_lp()` — sets the pending flag and calls `cond_yield_lp()`, which performs the register/stack switch if another thread is ready.

`abort()` removes a returning thread from the scheduler and moves it to `DEAD_THREADS` (a deferred reaper list). The reaper drops the thread from a different thread's context (after the context switch), so the thread's own kernel stack (which is freed on drop) is no longer in use.

2.5 Inter-processor interrupts (IPIs)

The `IPI_CMD_QUEUES` are per-LP bounded `ConcurrentQueue` queues (256 entries by default). Sending a cross-LP work item pushes an RPC to the target LP's queue and delivers an IPI. The target's IRQ dispatcher drains the queue in order.

`IpiRpc` carries kernel-internal RPCs (TLB shutdown, scheduler wakeup) and a `Closure(Box<dyn FnOnce() + Send>)` variant for arbitrary cross-LP work dispatch — the seed of `sitas`'s typed `ShardMailbox<M>`.

Two push paths:

- `try_push_to / try_send_ipi_rpc` — returns `Err (rpc)` on full queue (first-class backpressure).
- `push_to / send_ipi_rpc` — non-fallible force-push for must-not-drop kernel RPCs. Evicts the oldest entry if full.

2.6 Capability tables

A per-address-space `IdTable<Arc<Completion>>` maps a small integer `CompletionCap` (an index into the table) to a kernel object naming an in-flight or completed operation. The table is bounded (submission backpressure: `submit` returns `WouldBlock` when full). `Completion` is `Observable`: a waiting thread's `Waker` is registered on it via `completion::wait (asid, cap)`.

3 Execution model

This chapter describes how threads run, block, and yield — the runtime that the async-syscall ABI and the sitas reactor operate on top of.

3.1 Thread lifecycle

A thread is created with `spawn_thread(asid, entry_point)`. The `asid` determines whether it is a kernel thread (`asid == 0`, runs at EL1 / ring 0) or a user thread (`asid > 0`, drops to EL0 via `user_trampoline`).

3.1.1 Entry

The trampoline unmask interrupts, calls the entry function, then calls `abort()` when the entry returns. `abort()` removes the thread from the scheduler and moves it to `DEAD_THREADS`; the reaper drops it (freeing its stack) from the next thread's context, after the context switch.

3.1.2 Blocking

A thread blocks by calling `block_thread(tid, &observable)` through a higher-level primitive (`sleep`, `wait`). The call marks the thread `Blocked`, registers its `Waker` as an observer on the event, removes it from the LP's run queue, and `yield_lp()` saves the thread's execution context and switches away. When the event fires, the waker submits the thread back to the run queue; the thread resumes exactly where it blocked.

3.1.3 Waking

`Waker::notify(tid)` calls `submit_ready_thread(tid)`, which adds the thread to the least-loaded LP's run queue and sends a wakeup IPI if that LP is idle and is not the calling LP. The thread runs at the next scheduling opportunity.

3.1.4 Yielding

`yield_lp()` sets the context-switch-pending flag and calls `cond_yield_lp()`. If the flag is set and another thread is runnable, the current thread's callee-saved registers and stack pointer are saved to its `ThreadContext`, and the next thread's are restored.

3.2 The quantum timer

Each LP's `RoundRobin` scheduler arms a periodic quantum `TimerEvent` (10 ms by default). When the timer fires, the observer sets the pending flag; the next `yield_lp()` (or the IRQ dispatcher's tail) performs the context switch.

The quantum is re-armed after each context switch (or when the flag is cleared with nothing to switch to). An armed flag ensures exactly one quantum event is in flight at any time, preventing unbounded timer-queue growth from manual yields. The timer follows the same block-wake pattern as any other event: the hardware timer fires, the IRQ dispatcher processes the queue, `LpTimer::start()` re-arms for the next deadline.

3.3 The block-yield-wake cycle

This is the fundamental loop of the async runtime:

1. **Worker thread** calls `sleep(50ms)`. The timer event is created, the thread is blocked on it, the event is enqueued in the per-LP timer queue, the hardware timer is armed for the nearest deadline, and `yield_lp()` switches away.
2. **Other threads** run (or the LP idles). The timer IRQ fires at each quantum; the event-queue processes expired events.
3. **50 ms elapse**. The sleep event is at the front of the timer queue. `process_events` pops it, calls `signal()`, which upgrades the observer `Weak` and calls `Waker::notify()`, which calls `submit_ready_thread(tid)`.
4. **Worker** is now `Ready`. At the next scheduling point it runs, resuming immediately after its `yield_lp()` inside `sleep()`.

The block-wake cycle for `completion::wait` is identical: `complete()` signals the completion's observers (instead of a timer signal), waking the blocked thread.

3.4 Context saving

`switch_ctx(curr_sp_ptr, next_sp_ptr)` is the architecture-specific register-and-stack swap (AArch64: `x19-x30` and `TTBR0_EL1`; x86-64: callee-saved GPRs and `CR3`). It is called with both scheduler locks released, because it may permanently abandon the current stack (on the initial switch away from boot context). The post-switch code reaps dead threads.

4 The async syscall ABI

This chapter describes the userspace–kernel boundary for asynchronous system calls. It is the concrete interface that `sitas` (the shard-per-core userspace runtime) binds to, and it is the design surface where both projects converge. Every concept described here is implemented and verified on QEMU AArch64 and x86-64.

4.1 The five operations

4.1.1 `submit` — start an async operation

`submit(asid, op_code, buffer) -> Result<CompletionCap, SubmitError>`. Returns immediately with a `CompletionCap` naming the in-flight operation. Ownership of the buffer (an owned `Vec<u8>`) transfers to the kernel for the operation’s duration. Returns `SubmitError::WouldBlock` when the capability table is full (submission backpressure).

4.1.2 `wait` — the reactor’s only sleep

`wait(asid, cap) -> Result<(), CapError>`. Blocks the calling thread until the specified capability reaches terminal completion. Internally registers the thread’s `Waker` as an observer of the `Completion` (which is `Observable`) and yields.

4.1.3 `poll` — non-blocking check

`poll(asid, cap) -> Result<Option<Completed>, CapError>`. Returns `Ok(None)` while in flight; returns `Ok(Some(Completed { result, buffer }))` when terminal. The buffer ownership returns to the caller. Idempotent after completion (returns `None` once drained).

4.1.4 `cancel` — drop-as-cancellation

`cancel(asid, cap) -> CancelState`. Requests cancellation of an in-flight operation. If the operation is still in flight, returns `CancelRequested`: the kernel retains any transferred buffer until it posts a terminal `Cancelled` completion (deferred reclaim, mirroring `sitas`’s `defer_buffer_drop`). If the operation already completed, returns `AlreadyComplete`.

4.1.5 `close` — release a slot

`close(asid, cap) -> Result<(), CapError>`. Frees a capability-table slot after its terminal completion was posted or its result was consumed by `poll`. Fails with `NotComplete` if the operation is still in flight.

4.2 The exit-observer completion path

The ABI's intended completion mechanism is declarative: **the worker thread exiting IS the completion event**. `submit_worker(asid, worker_entry, result)` spawns a worker thread and registers the returned capability as the worker's exit-observer. The worker performs its work and returns; the thread's `Thread::Drop` fires the exit-observer, which calls `complete()` and wakes any waiter. The worker never references the capability explicitly.

This is the exact design described in the kernel's own code (`scheduler/threads/mod.rs:63-69`) and is implemented in `crates/catten/src/completion/mod.rs` (`CompletionExitObserver`).

4.3 The CQ ring (completion queue)

A shared-memory `io_uring`-style ring for zero-syscall completion delivery. A single 4 KiB page contains a header (head, tail, capacity, overflow) and a circular array of entries (16 bytes each: `cap: u64, result: i64`).

- **Producer (kernel):** `write(cap, result)` advances the head. `complete()` writes to the ring alongside the observer signal.
- **Consumer (userspace):** `read()` advances the tail, returns `Option<CqEntry>`. Zero syscalls.
- **Overflow:** when the ring is full (`head + 1 == tail modulo capacity`), the write is dropped and the overflow counter is incremented (non-lossy detection).

The ring is allocated from a physical frame and mapped into the user address space via `PageType::UserData` (AP_EL0 accessible). The same physical memory is accessible from the kernel via HHDM. Currently demonstrated in kernel-side self-tests; userspace mapping is proven to work (the real-EL0 test maps a CQ ring alongside code and result pages).

4.4 Cancellation and the buffer-reclaim contract

When userspace drops a `CompletionCap` without awaiting it (or calls `cancel`):

1. The kernel marks the operation cancelling but **retains the buffer** — it may still be the target of DMA or kernel access.
2. A terminal `Cancelled` completion is eventually posted to the CQ ring, handing the buffer back.
3. On teardown (address-space exit), outstanding buffers are leaked rather than freed while the kernel may touch them (the kernel-side mirror of `sitas's mem::forget-on-timeout` rule).

This is not Unix-specific; it is an ownership contract. The ABI is specified against `sitas's io_uring` buffer-ownership discipline, and `sitas's` existing tests (`charlotte_abi.rs`) validate it.

4.5 Backpressure

- **Submission backpressure:** the per-AS capability table is bounded. `submit` returns `SubmitError::WouldBl` when full. The caller must drain completions (freeing slots) before retrying.

- **Completion backpressure:** the CQ ring is bounded. When full, the kernel does **not** drop completions; it stops producing new ones for that shard and marks the queue overflow-pending.
- **Cross-LP backpressure:** IPI_CMD_QUEUES are bounded per target LP. `try_push_to` returns `Err(rpc)` on full queue. This matches sitas's bounded `ShardSender<M>` and the cache-coherence analysis in its architecture doc.

5 Programming model

This chapter describes the rules, patterns, and primitives for writing kernel code that respects the async-first, shard-per-core model. These invariants are derived from `sitas` and are enforced by the kernel’s own concurrency design.

5.1 Invariants

5.1.1 Only the owning LP mutates its state

Per-LP data is accessed only from the LP that owns it. Cross-LP mutation must go through message dispatch (`ShardMailbox<M>`, IPI closure, or IPI RPC), never through a shared lock acquired from a foreign LP.

5.1.2 Messages are owned values

Everything that crosses a shard boundary is an owned value — a `Box<dyn FnOnce>`, an `IpiRpc`, or a typed `M` in a `ShardSender<M>`. No shared references, no `Arc<Mutex<...>>` as the normal programming model. `Send + 'static` boundaries enforce this at compile time.

5.1.3 References to shard-local state must not escape

`ShardLocal<T>::with(fn)` provides `&mut T` inside a closure; the borrow is verified at runtime (owner-check + re-entrancy flag). The closure must not store the reference, send it to another thread, or cross an `.await`. The same rule applies to `sitas`’s `ShardLocal<T>` userspace equivalent.

5.1.4 Work stays on its assigned LP

A thread spawned on LP `N` stays on LP `N`. To submit work to another LP, use `try_run_on_lp(target, closure)` or `ShardSender::try_send`. The kernel’s IPI infrastructure and the `sitas` `ShardedSubmitter` are the two sides of this contract.

5.1.5 Observability returns owned snapshots

The kernel’s house style is owned values for diagnostics. `sitas`’s `RuntimeSnapshot / ShardSnapshot` pattern applies: never expose borrowed references into runtime internals. Self-tests and the demo illustrate this.

5.2 Primitives

5.2.1 `ShardLocal<T>`

Lock-free per-LP container. Use for single-LP, thread-only data. Access: `sl.with(|val| { *val = 42; }).`

5.2.2 `ShardMailbox<M>`

Typed bounded per-LP queue. Cloneable `ShardSender<M>` (any LP can send), single-consumer `ShardReceiver<M>` (one per LP). `try_send(m)` returns `Err(m)` on backpressure.

5.2.3 IPI closure

`try_run_on_lp(target_lp, || { ... })`. Boxes arbitrary work and delivers it to the target LP's IPI queue. Returns the closure on full queue.

5.2.4 Completion API

`submit`, `submit_worker`, `complete`, `poll`, `wait`, `cancel`, `close`. The exit-observer path (`submit_worker`) is the intended default; the explicit `complete` is the escape hatch for non-thread-based work.

5.2.5 CQ ring

`CompletionQueueRing` for zero-syscall completion draining. Kernel side: `write(cap, result)`. Consumer side: `read() -> Option<CqEntry>`. Integration: `open_as_with_cq` attaches a ring to a capability table; `complete()` writes to it.

5.3 Rule-of-thumb guidance

- **Use** `PerLp<T>` for interrupt-accessed data and cross-LP access; **use** `ShardLocal<T>` for single-LP, thread-only data.
- **Use** `ShardMailbox<M>` (or `try_run_on_lp`) for cross-LP communication; do **not** lock an LP's state from another LP.
- **Use** `submit_worker` for fire-and-forget async kernel work; the worker just returns, and the exit-observer completes the capability.
- **Register** a CQ ring when creating an address space so that completions are visible to userspace without per-entry syscalls.
- **Accept** backpressure: bounded tables and queues return `WouldBlock / Err(m)`; retry or apply backpressure upstream. Never silently drop messages.

6 Development

6.1 Build targets

The kernel builds for three architectures via custom target JSON specs in `target_specs/`:

- `aarch64-unknown-none-catten.json`
- `x86_64-unknown-none-catten.json`
- `riscv64gc-unknown-none-catten.json` (planned, not actively maintained)

Build command (headless, no display dependency):

```
cargo build --package catten \
  --target target_specs/aarch64-unknown-none-catten.json \
  --no-default-features --features acpi
```

The display feature requires a C library for the flatterm framebuffer terminal; headless boot uses serial output only (AArch64: PL011 UART; x86-64: 16550 COM1). AArch64 boots via `scripts/boot-smp1.sh` (single-core) or `scripts/boot-aarch64.sh`. x86-64 boots via `scripts/boot-x86.sh` (requires `-cpu max` for RDTSCP/FSGSBASE).

6.2 Self-tests

There is no host-side `cargo test`; all tests are **boot-time kernel self-tests** (~whitebox integration tests) run from `self_test::run_self_tests()` in `main.rs:104`, called after `INIT_BARRIER.wait()`. Each test uses `assert!()` / `assert_eq!()` (panic = failure). Current test suite (both arches pass):

- `self_test/completion.rs` — buffer ownership, cancel / deferred-reclaim, backpressure
- `self_test/syscall.rs` — dispatch table routing via synthetic `TrapFrame`
- `self_test/ipi.rs` — bounded queue, backpressure, closure dispatch
- `self_test/shard.rs` — `ShardLocal<T>` owner-check / re-entrancy guard, `ShardMailbox<M>` send/recv
- `self_test/el0.rs` — real-EL0 user thread (AArch64 only)
- `self_test/cq.rs` — CQ ring write/drain/overflow
- `self_test/cq_completion.rs` — CQ ring integrated with completion subsystem

6.3 Async-syscall demo

`crates/catten/src/demo.rs` spawns a coordinator + worker thread from `bsp_main` (after the scheduler is active). The coordinator calls `submit_worker`, which spawns a worker and returns a capability that fires when the worker exits; the coordinator blocks on `wait(cap)`. The worker sleeps, returns,

and the exit-observer completes the cap, waking the coordinator. This exercises the full submit-async-work-complete-wake loop.

Output at boot:

```
Testing Complete. All Tests Passed!
[async-demo] coordinator: submitted worker, now awaiting completion...
[async-demo] worker: performing async work (sleep 50ms)
[async-demo] worker: work finished, exiting (completion fires on exit)
[async-demo] coordinator: COMPLETION RECEIVED, result Ok(42)
[async-demo] SUCCESS: async syscall round-trip complete (via thread-exit
                observer)
```

6.4 CI

GitHub Actions (`.github/workflows/ci.yml`) builds both architectures with `-D warnings` and runs an automated QEMU boot gate on the dev branch (grepping for “Testing Complete. All Tests Passed!”).

6.5 Where to start reading

- `crates/catten/src/completion/mod.rs` — the completion subsystem (capability table, submit/complete/poll/wait/cancel/close, `submit_worker`, `exit-observer`).
- `crates/catten/src/completion/cq.rs` — the CQ ring buffer.
- `crates/catten/src/syscall/mod.rs` — AArch64 syscall dispatch table.
- `crates/catten/src/demo.rs` — the end-to-end async demo.
- `crates/catten/src/cpu/scheduler/` — `threads`, `wakers`, `RoundRobin`, `block_thread`.
- `crates/catten/src/cpu/multiprocessor/ipi.rs` — bounded IPI queues and the `IpiRpc` enum.
- `crates/catten/src/cpu/scheduler/threads/mod.rs:63-69` — the completion-capability design comment.

7 Current status

7.1 Verified (boots on QEMU, zero panics)

- **AArch64:** boots, all self-tests pass (`Testing Complete. All Tests Passed!`), the async-syscall demo succeeds with `Ok(42)` via exit-observer, real-EL0 user thread executes and reads the shared CQ ring.
- **x86-64:** boots with `-cpu max`, all self-tests pass, the async-syscall demo succeeds with `Ok(42)` via exit-observer. (x86-64 ring-3 / SYSCALL userspace is **not** yet exercised.)
- **Completion subsystem:** `submit`, `complete`, `poll`, `wait`, `cancel`, `close`, `submit_worker` (exit-observer), CQ ring, CQ ring integrated with AS.
- **Syscall dispatch:** AArch64 SVC handler decodes EC, builds `TrapFrame`, dispatches to handler table. x86-64 SYSCALL trampoline exists but is not yet connected to the dispatch table.
- **IPI:** bounded per-LP queues, backpressure, typed closure dispatch, AArch64 and x86-64 unstubbed.
- **ShardLocal<T> / ShardMailbox<M>:** implemented and tested.
- **Scheduler:** RoundRobin, quantum timer, block/yield/wake, deferred reaper.
- **CI:** GitHub Actions builds both arches, QEMU boot gate.

7.2 Known limitations

- **x86-64 ring-3 userspace:** SYSCALL entry trampoline exists, GS base and IRETQ return path remain to be wired (see `docs/x86_64-bringup-status.md`). The EL0 test and the real-EL0 user thread are AArch64-only.
- **sleep / preemption:** fully fixed on AArch64; re-arming and quantum-guard are correct. The earlier heisenbugs (timer stops, bounded growth) are resolved.
- **Guard-page overlap:** adjacent kernel stacks share guard pages (`find_free_region` doesn't account for unmapped guards); reference-counting in the stack allocator prevents leaks. A stack freed while adjacent to a live one is handled correctly.
- **Multi-LP:** infrastructure exists but `-smp > 1` has not been exercised. AP startup, per-LP GDT/TSS/IDT, and cross-LP IPI work correctly in single-LP tests; multi-LP would be the next scaling step.
- **CQ ring userspace mapping:** the CQ ring is allocated on a physical frame and the real-EL0 test proves mapping a shared page into a user AS; wiring the ring for actual userspace consumption is deferred.

7.3 Prior art / independent verification

The sitas repository (`reactor-handle-seam` branch) contains an in-memory reference model of the CharlotteOS ABI (`src/charlotte_abi.rs`) with 9 tests validating the contract shapes against the `ReactorBackend` trait. This model was built before the kernel prototype existed and independently verified the ABI design.

8 The Collaboration: A Retrospective

8.1 The convergence

Two independent projects started from opposite ends of the software stack and built toward the same shape, without coordination.

8.1.0.0.1 `sitas`

began in userspace, asking: *what does shared-nothing, shard-per-core concurrency look like when it is built out of Rust ownership instead of locks?* Its answer was a discipline — each shard owns its state, only the owning shard mutates it, everything crossing a shard boundary is an owned value moved through an explicit typed message. On top of that it grew a custom async executor, per-shard reactors, `io_uring` completion I/O, and snapshot-based observability.

8.1.0.0.2 `CharlotteOS`

began in the kernel, asking: *hardware is asynchronous with respect to the CPU, so why is the OS synchronous?* Its answer was to make the kernel async-first — cheap threads, an observer/waker event model, and system calls that are asynchronous by default, returning a completion capability that userspace can wait on.

Both projects independently converged on the same primitive: a waitable completion handle. `sitas` called it `Reply<T> / Notify`; `CharlotteOS` called it `Observer / Waker`. They are the same object with different names.

The insight that drove the collaboration: **a userspace runtime whose async model agrees with the kernel it runs on removes the “retrofit tax”** — the async-signal-safety, thread pools, and `io_uring`-over-blocking emulation that exist only because the runtime and the OS disagree about what is fundamental.

8.2 The structural correspondence

The reason the projects belong together is deeper than analogy: the pieces already existed on both sides, and they lined up almost one-to-one.

`sitas` was, in effect, re-deriving in userspace the runtime that `CharlotteOS` was trying to be natively. Where `sitas` fought to build a reactor on top of a synchronous OS, `CharlotteOS` offered the reactor as the substrate.

8.3 The exploration phases ($A \rightarrow C \rightarrow B$)

The collaboration was structured as three time-boxed spikes, pursued in dependency order, each building on the prior. Nothing was merged until it earned its place. All spikes were verified on QEMU (AArch64 and, later, x86-64).

sitas concept	Role	CharlotteOS equivalent
Shard = OS thread pinned to core	Unit of isolation	Logical Processor (LP) + PerLp
ShardId explicit placement	Addressing	LpId
Shard mailbox (ShardSender/ShardReceiver)	Cross-shard transport	IPI_CMD_QUEUES (bounded p
Custom executor Waker re-enqueue	Wake a task	Waker (ThreadId) + submit
Reply<T> / Notify awaitable completion	Await a result	Observable / Observer + T
Reactor idle wait	Block until events	GIC/APIC IRQ dispatch → con
io_uring completion I/O	Async syscalls	Completion-capability ABI (desi
ShardLocal<T> (no mutex, owner-checked)	Shard-owned mutable state	ShardLocal<T> in-kernel (sit
CPU pinning	Bind shard to core	LP is a core (not advisory)

8.3.1 Phase A — OS backend seam (sitas)

Goal: extract the reactor’s OS dependency into an explicit trait, so the executor can be backed by anything that can block until events and wake from elsewhere — not just Unix `epoll/kqueue`.

Deliverable: the `ReactorBackend` trait (in sitas’s `reactor_backend.rs`). Three methods capture everything the executor needs from the OS:

- `waker()` — obtain a cloneable waker.
- `wait(read, write, timeout) → Event` — block until an interest becomes ready, the reactor is woken, or a deadline elapses.
- `wake()` — wake a blocked reactor from another thread/core.

A mock in-memory backend proved a non-Unix backend is implementable behind the same interface. The existing Unix `OsReactor / OsWaker / OsEvent` satisfied the contract with zero changes.

Key finding: abstracting “block until an event” was trivial. Abstracting “*what is a waitable interest*” was the real work. On Unix the interest is a `RawFd`; the trait’s `Handle` associated type is where the design pressure concentrates. For CharlotteOS, `Handle` is a completion capability.

8.3.2 Phase C — Async syscall ABI (CharlotteOS kernel)

Goal: design the userspace↔kernel boundary for asynchronous system calls, using sitas as the executable specification. If sitas’s shard model, completion futures, and cross-shard submit map cleanly onto the ABI, the ABI is right.

Deliverable: five kernel-internal operations that later became the syscall table:

1. `submit(op, args, buffers) → CompletionCap` (a per-address-space handle naming in-flight work).
2. `wait(cq, min_complete, deadline) → count` (the reactor’s only sleep).
3. `wake(cq)` (cross-shard wake via IPI).
4. `cancel(cap)` (deferred buffer reclaim).
5. `close(cap)` (release a completed slot).

Each built on facilities that already existed in the kernel: the per-AS `IdTable` for the capability table, the `Observable/Observer` mechanism for completing operations, the per-LP `RoundRobin` scheduler for blocking and waking threads, and the bounded `IPI_CMD_QUEUES` for cross-core communication.

Key infrastructure built in this phase:

- **Completion-capability table** (`completion/mod.rs`) — per-address-space `IdTable<Arc<Completion>`», bounded for submission backpressure.
- **CQ ring** (`completion/cq.rs`) — a shared-memory `io_uring`-style ring buffer for zero-syscall completion delivery. Single 4 KiB page, header + entry array. The kernel writes entries; userspace reads them without syscalls.
- **Syscall dispatch** (`syscall/mod.rs`) — the `AArch64 sync_dispatcher` decodes `ESR_EL1.EC` and routes `SVC` instructions to a dispatch table. A `TrapFrame` captures the user register state.
- **Bounded IPI queues** — `IPI_CMD_QUEUES` became bounded per-LP `ConcurrentQueue<256>` with first-class backpressure (`try_push_to` returns `Err(rpc)` on full).
- **`IpiRpc::Closure`** variant — arbitrary `FnOnce()` + `Send` work dispatched to a target LP, the seed of `ShardMailbox<M>`.

8.3.3 Phase B — sitas’s discipline inside the kernel

Goal: adopt sitas’s shared-nothing invariants as the kernel’s own concurrency doctrine. CharlotteOS already had the ingredients; sitas contributed the rules.

Deliverables:

- **`ShardLocal<T>`** — a lock-free per-LP container replacing `PerLp<T>`’s `RwLock` with `UnsafeCell<T>` and two runtime assertions: owner-check (the caller must be on the owning LP) and re-entrancy guard (borrow flag). Closure-based access ensures `&mut T` never escapes.
- **`ShardMailbox<M>`** — a typed, bounded per-LP queue with cloneable `ShardSender<M>` and single-consumer `ShardReceiver<M>`, built on the bounded IPI infrastructure. `try_send` returns `Err(m)` on backpressure.

8.3.4 Beyond the plan — what was built additionally

- **Exit-observer completion** — the ABI’s intended mechanism: `submit_worker` registers a completion capability as the worker thread’s exit-observer. The thread exiting *is* the completion event. This required fixing the thread lifecycle (deferred reaper) so threads could return cleanly.
- **Real-EL0 user thread** — the first userspace thread running at EL0, invoking `SVC` from compiled Rust code, with the kernel dispatching the syscall and returning.
- **x86-64 parity** — serial console (16550 UART), SYSCALL entry trampoline, unstubbed IPI handlers, LA57 paging-mode fix. The same self-test suite passes on both architectures.
- **16550 UART serial driver** — x86-64 headless boot was completely silent before this; debugging was impossible.
- **Multi-LP bring-up** — kernel boots with `-smp 2` on `AArch64` (cross-core SGI delivery is blocked by a QEMU GICv3 emulation defect; self-targeted SGIs and per-core timers work on all LPs).
- **sitas-core `no_std` refactor** — the sitas executor, reactor traits, `ShardLocal`, `ShardMailbox`, and completion primitives were split from the Unix backend into a `no_std + alloc` crate. A lock-free `ringbuf` replaced `concurrent-queue`. The `ShardRuntime` trait abstracts thread spawning.

- **sitas-charlotte backend** — implements `ReactorBackend` and `ShardRuntime` against the kernel's async syscall ABI. The user binary links `sitas-core` + `sitas-charlotte` and runs `ShardedExecutor` on `CharlotteOS`.

8.4 The async syscall ABI in detail

The ABI is the concrete interface where both projects meet. It is specified against `sitas`'s `ReactorBackend` trait and implemented in the `CharlotteOS` kernel.

8.4.1 The five operations

submit(asid, op, buffer) → CompletionCap Returns immediately with a capability naming the in-flight operation. Ownership of the buffer transfers to the kernel until a terminal completion posts it back. Returns `SubmitError::WouldBlock` on submission backpressure (capability table full).

wait(asid, cap) → Result<(), CapError> Blocks the calling thread until the specified capability reaches terminal completion. Internally registers the thread's `Waker` as an `Observer` of the `Completion` (which is `Observable`) and yields.

poll(asid, cap) → Option<Completed> Non-blocking check. Returns `None` while in flight; returns `Some(Completed { result, buffer })` when terminal. Buffer ownership returns to the caller.

cancel(asid, cap) → CancelState Requests cancellation. If in flight, returns `CancelRequested`; the kernel retains any transferred buffer until it posts a terminal `Cancelled` completion (deferred reclaim). If already complete, returns `AlreadyComplete`.

close(asid, cap) → Result<(), CapError> Frees the capability-table slot after the terminal completion was posted or the result was consumed by `poll`. Fails with `NotComplete` for in-flight operations.

8.4.2 The exit-observer completion path

The ABI's intended completion mechanism is declarative: **the worker thread exiting IS the completion event**. `submit_worker(asid, worker_entry, result)` spawns a worker thread and registers the returned capability as the worker's exit-observer. The worker performs its work and returns; the thread's `Thread::Drop` fires the exit-observer, which calls `complete()` and wakes any waiter. The worker never references the capability explicitly.

This matches the design comment in `scheduler/threads/mod.rs:63-69` and is implemented as `CompletionExitObserver` in `crates/catten/src/completion/mod.rs`.

8.4.3 The CQ ring

A shared-memory `io_uring`-style ring buffer for zero-syscall completion delivery. A single 4 KiB page contains a header (head, tail, capacity, overflow) and a circular array of entries. The kernel (producer) writes entries via `write(cap, result)`; userspace (consumer) drains via `read() → Option<CqEntry>` – zero syscalls.

8.4.4 Cancellation and the buffer-reclaim contract

When userspace drops a `CompletionCap` without awaiting it:

1. The kernel marks the operation cancelling but **retains the buffer** — it may still be the target of DMA or kernel access.
2. A terminal `Cancelled` completion is eventually posted, handing the buffer back.
3. On teardown, outstanding buffers are leaked rather than freed while the kernel may touch them.

This is the kernel-side mirror of `sitas's defer_buffer_drop` and `mem::forget-on-timeout` rule.

8.4.5 Backpressure

- **Submission:** the per-AS capability table is bounded. `submit` returns `WouldBlock` when full.
- **Completion:** the CQ ring is bounded. When full, the kernel does *not* drop completions; it stops producing and marks the queue overflow-pending.
- **Cross-LP:** `IPI_CMD_QUEUES` are bounded per target LP. `try_push_to` returns `Err(rpc)` on full.

8.5 The bugs we found (and the real value of running on hardware)

Bootstrapping an async-first kernel on real hardware (QEMU) surfaced a series of latent bugs that static analysis and self-tests never caught. Each fix enabled the next layer of work. The following are the most instructive, in the order they were discovered.

8.5.1 Panic handler recursed through heap-dependent `println`

Symptom: any fault during early boot (before the heap allocator was ready) silently triple-faulted instead of printing a panic message — making *every* early bug look like a mysterious hang.

Root cause: the panic handler used `println!`, which routes through `INT_STATE` — a `LazyLock` whose first-use initialization allocates from the kernel heap (`alloc::vec!`). A fault before the heap was ready caused the panic handler's `println!` to trigger a heap allocation → another fault → re-entered the panic handler → infinite recursion → stack overflow → triple fault.

Fix: the panic handler now uses `early_println!` (direct serial writes, no heap, no `INT_STATE`). A panic handler must never depend on the allocator it might be reporting a failure about.

This was the single most valuable fix of the entire project: it unmasked every subsequent fault as a visible panic instead of a silent triple-fault.

8.5.2 Blocked threads lost their execution context

Symptom: a thread that blocked in `wait()` (or `sleep()`) restarted from its entry point when woken, instead of resuming where it blocked.

Root cause: `block_thread` cleared the LP scheduler's `current_handle` for the running thread, so the subsequent `cond_yield_lp` read `curr_tid == None` and `switch_ctx(null, ...)` **skipped** saving the blocking thread's execution context.

Fix: a self-blocking thread now stays `current_handle` until the yield saves its context; `RoundRobin::next` declines to re-queue a `Blocked` thread. This was a foundational fix: `wait()` and `sleep()` had never been exercised before.

8.5.3 RwLock held-across-call deadlocks in `sleep()` and `abort()`

Symptom: `sleep()` worked reliably with diagnostic logging but hung without it — a classic heisenbug.

Root cause: `sleep()` held a `SYSTEM_SCHEDULER.read()` guard (plus the LP scheduler lock) across `SYSTEM_SCHEDULER.write().block_thread(...)` — a read-then-write `RwLock` self-deadlock. The diagnostic version happened to bind the thread ID first, releasing the guard. `abort()` had the same pattern with the LP scheduler lock.

Fix: bind the value out of the scrutinee first so the temporary guards drop before the re-locking call.

8.5.4 Quantum timer grew the timer queue without bound

Symptom: after extended operation, the per-LP timer queue contained thousands of quantum events, consuming unbounded memory.

Root cause: `clear_ctx_switch_pending` enqueued a new quantum `TimerEvent` on every yield, not just on quantum expiry. Manual yields accumulated quantum events forever.

Fix: an armed flag keeps exactly one quantum event in flight (the quantum's own firing clears it). Also, `cond_yield_lp` re-arms even when there is nothing to switch to (previously the timer stopped when only one thread was runnable), and `process_events` stops the level-triggered timer when the queue drains.

8.5.5 A thread cannot free its own kernel stack

Symptom: a returning kernel thread hung during teardown.

Root cause: `ThreadContext::Drop` calls `deallocate_stack`, which frees the kernel stack — but the returning thread is still executing on that stack when `abort` drops it.

Fix: a deferred reaper. `abort` moves the dying thread to `DEAD_THREADS` (`IdTable::take_element` extracts it without dropping), and `cond_yield_lp` calls `reap_dead_threads()` *after* switching away, so the stack is freed from a different thread's context.

8.5.6 Stack allocator never registered guard pages

Symptom: `deallocate_stack` always failed with `InvalidStack`, causing the reaper to leak stacks.

Root cause: `allocate_stack` never populated `KERNEL_GUARD_PAGE_SET`, so `validate_stack` couldn't find the guards.

Fix: register lower and upper guard pages on allocation, and rewrite `deallocate_stack` to derive the page count from them. Reference-count guard pages (using `BTreeMap<VAddr, usize>`) so adjacent stacks sharing a guard survive until both are freed.

8.5.7 LA57 paging-mode mismatch on x86-64

Symptom: a #GP with `RAX = 0xff90000000000000` (clearly non-canonical under 4-level paging) during kernel-heap large-page mapping.

Root cause: `CpuInfo::get_vaddr_sig_bits()` read the CPU's *maximum* linear address width from CPUID 0x80000008 EAX[15:8] – with `-cpu max` that is 57 (the CPU supports 5-level paging). The kernel then selected `LA_MAP_57BIT`, whose `kernel_allocator_arena` base is `0xff90000000000000` – canonical only under 5-level paging. But Limine boots with 4-level paging (48-bit VAs), under which that address is non-canonical.

Fix: derive the address width from `CR4.LA57` (bit 12) — the *active* paging mode, not the CPU's capability.

8.5.8 ASID-0 collision (“the TLB bug that wasn’t”)

Symptom: the first user thread faulted at its entry point with an instruction abort (`EC=0x21`, “no change in exception level”).

Root cause: the global `ADDRESS_SPACE_TABLE` started empty, so the first user address space got id 0, which equals `KERNEL_ASID`. `Thread::new` then built a *kernel* thread context, running the user code at EL1 where `PXN=1` forbids execution. The EC decode (`0x21` = fault without EL change) was the tell — the code was never at EL0.

Fix: pre-populate the table with the kernel AS at id 0, so user spaces always get non-zero ids.

A second bug in the same test: `sync_dispatcher` added 4 to `ELR_EL1` on SVC, but `AArch64` already sets ELR to the instruction *after* SVC. The +4 overshoot the loop instruction, landing in zeroed memory (`EC=0` undefined instruction).

8.5.9 x86-64 kernel_thread_trampoline halted on thread return

Symptom: on x86-64, the async demo's worker thread completed its work but the coordinator never resumed.

Root cause: the x86-64 `kernel_thread_trampoline` entered a `hlt` loop when a thread's entry returned, instead of calling `abort()` like the `AArch64` trampoline. The CPU halted, freezing the LP.

Fix: make the x86-64 trampoline call `abort` on entry return, matching `AArch64`'s behavior. (The `AArch64` trampoline had always called `abort` via `bl abort` after the entry point returned.)

8.6 Verification

All verification is via QEMU emulation (`AArch64 virt` and `x86-64 q35` machines, TCG accelerator). The kernel has no host-side test framework; all tests are boot-time **kernel self-tests** that run from `main.rs:104` via `self_test::run_self_tests()`.

8.6.1 Self-test suite (both architectures pass)

- `self_test/completion.rs` — buffer ownership, cancel / deferred-reclaim, backpressure, close rejection
- `self_test/syscall.rs` — dispatch table routing via synthetic `TrapFrame` (all 7 syscalls routed without panicking)
- `self_test/ipi.rs` — bounded queue, backpressure, closure dispatch
- `self_test/shard.rs` — `ShardLocal<T>` owner-check / re-entrancy guard, `ShardMailbox<M>` send/recv
- `self_test/el0.rs` — real-EL0 user thread (AArch64 only), compiled Rust binary calling SVC with correct dispatch
- `self_test/cq.rs` — CQ ring write/drain/overflow detection
- `self_test/cq_completion.rs` — CQ ring integrated with completion subsystem

8.6.2 Async-syscall demo (both architectures)

The `demo.rs` spawns a coordinator + worker from `bsp_main`. The coordinator calls `submit_worker`, which spawns a worker and registers the capability as the worker's exit-observer. The worker performs async work (`sleep 50ms`) and returns; the thread-exit fires the observer, completing the capability and waking the coordinator. Verified output:

```
Testing Complete. All Tests Passed!
[async-demo] coordinator: submitted worker, now awaiting completion...
[async-demo] worker: performing async work (sleep 50ms)
[async-demo] worker: work finished, exiting (completion fires on exit)
[async-demo] coordinator: COMPLETION RECEIVED, result Ok(42)
[async-demo] SUCCESS: async syscall round-trip complete (via thread-exit observer)
```

8.6.3 sitas integration

The user binary (`crates/catten-user/`) compiles against `sitas-core` (`no_std` executor) and `sitas-charlotte` (ReactorBackend against kernel syscalls). The binary creates a `ShardedExecutor` with `CharlotteReactor` as the `ShardRuntime`, spawning a shard thread via SVC #7 (`SPAWN_THREAD`). The spawned thread writes a sentinel to the shared result page. Binary size: 144 bytes (up from 48 without sitas). Zero panics.

8.6.4 CI

GitHub Actions (`.github/workflows/ci.yml`) builds both architectures with `-D warnings`, and runs an automated QEMU boot gate on the dev branch (60-second timeout, grepping for “Testing Complete. All Tests Passed!”).

8.7 What we learned

1. **The block-yield-wake cycle is the hard problem.** Designing the completion ABI was straightforward compared to making `wait()` correctly save and restore thread state through the cooperative

scheduler. Every scheduling primitive (`sleep`, `wait`, `abort`) exercised a new path and surfaced a new bug.

2. **Bootting on real hardware finds bugs that static analysis cannot.** The panic-handler recursion, ASID-0 collision, and LA57 paging mismatch were all invisible in self-tests and logical reasoning. Every one was found by bootting and observing the actual fault.
3. **The observer/waker pattern is the right primitive.** Once `Completion` was made `Observable` and `wait()` used the same path as `sleep()`, the entire completion subsystem fell into place. The exit-observer mechanism is the natural endpoint: a thread exiting *is* the completion event.
4. **The async model genuinely removes the retrofit tax.** On CharlotteOS, `wait(cap)` blocks exactly as `sleep(50ms)` blocks — both register a `Waker` on an `Observable` and `yield`. There is no emulation, no thread pool, no signal-handler tightrope. The kernel and the runtime agree about what is fundamental.
5. **The sitas traits were essential for co-design.** The `ReactorBackend` trait defined in Phase A became the executable specification for the async syscall ABI in Phase C. The `sitas-charlotte` backend validates the ABI shapes before any kernel path exists. This bidirectional validation is the core of the collaboration.

8.8 Current status (July 2026)

Verified on QEMU AArch64 and x86-64:

- Both architectures boot, all self-tests pass, exit-observer async demo succeeds.
- Completion subsystem: `submit`, `complete`, `poll`, `wait`, `cancel`, `close`, `submit_worker`, `exit-observer`, CQ ring.
- Syscall dispatch: AArch64 SVC handler (7 syscalls). x86-64 SYSCALL trampoline exists (connected to dispatch table).
- IPI: bounded per-LP queues, backpressure, typed closure dispatch.
- `ShardLocal<T>` / `ShardMailbox<M>` in-kernel.
- Scheduler: `RoundRobin`, quantum timer, `block/yield/wake`, deferred reaper, per-LP thread pinning.
- `sitas-core no_std` crate + `sitas-charlotte ReactorBackend` + `ShardRuntime::spawn_shard(SVC #7)`.
- CI: GitHub Actions builds both arches, QEMU boot gate.

Known limitations (documented, scoped):

- x86-64 ring-3 / SYSCALL userspace (GS base, IRETQ return) — not yet exercised.
- Full `ShardSet/Sharded` threading layer (the `ShardRuntime` trait needs to be threaded through the `ShardSet::start` call-chain; currently `ShardedExecutor` works directly but the higher-level service modules still use `thread::spawn`).
- Multi-LP cross-core IPI (QEMU GICv3 SGI delivery defect — self-IPIs and per-core timers work on all LPs).

8.9 Repositories and branches

- **sitas** (`reactor-handle-seam`): `ReactorBackend` trait, ABI reference model (`charlotte_abi.rs`), `sitas-core` `no_std` crate, `sitas-charlotte` crate.
- **CharlotteOS** (`dev`): consolidated kernel with completion subsystem, syscall dispatch, bounded IPI queues, `ShardLocal`, `ShardMailbox`, CQ ring, real-EL0 test, exit-observer completion, `SPAWN_THREAD` syscall, sitas user binary.

9 Glossary

AS (Address Space) A page-table context identified by an `AddressSpaceId` (0 = kernel, >0 = user). Contains the `TTBR0_EL1` (user) and `TTBR1_EL1` (kernel) pointers on AArch64, or `CR3` on x86-64.

Block (verb) To remove a running thread from the scheduler and register its `Waker` on an `Observable`. The thread yields; when the event fires the waker re-admits it.

CQ ring (Completion Queue) A shared-memory ring buffer for zero-syscall completion delivery from the kernel to userspace.

Capability table A per-AS `IdTable<Arc<Completion>>` mapping `CompletionCap` (index) to an in-flight or completed operation.

Completion A kernel object representing an in-flight or completed async operation. `Observable`: threads wait on it, complete signals it.

Exit-observer An `Observer` registered on a `Thread`; fires when the thread is dropped (exits). The ABI's intended completion mechanism: `submit_worker` registers the capability as the worker's exit-observer.

HHDM (Higher Half Direct Mapping) A linear mapping of all physical memory into the kernel's virtual address space, provided by the bootloader (Limine). `PAddr -> *mut T` goes through HHDM.

IdTable A sparse `Vec<Option<T>>` with ID recycling. Used for the thread table, address-space table, and capability table.

IPI (Inter-Processor Interrupt) A signal from one LP to another. The kernel's IPI queues are bounded per-LP `ConcurrentQueue` instances carrying `IpiRpc` values.

LP (Logical Processor) One hardware core. The unit of sharding in both the kernel and `sitas`.

Observable / Observer The event-notification pattern. An `Observable` holds `Weak<dyn Observer>` references; when it fires, it calls `Observer::notify()`.

PerLp<T> A boxed slice of `RwLock<T>`, one per LP. Interrupt-safe (masks interrupts on acquire).

Quantum timer The per-LP periodic timer (10 ms) that drives preemptive context switching in the `RoundRobin` scheduler.

Reaper The deferred-thread-cleanup mechanism: dead threads are moved to `DEAD_THREADS` and dropped from the next thread's context after the context switch.

ShardLocal<T> A lock-free per-LP container using `UnsafeCell<T>` with an owner-check and borrow-flag guard. `sitas`-derived.

ShardMailbox<M> A typed bounded per-LP queue with `ShardSender<M>` (cloneable) and `ShardReceiver<M>` (single-consumer). First-class backpressure. `sitas`-derived.

TrapFrame An architecture-specific snapshot of the user register state at the moment an exception was taken. Passed to the syscall handler on AArch64.

Waker An `Observer` wrapping a `ThreadId`. When notified, calls `submit_ready_thread(tid)`.

Yield To voluntarily give up the LP by calling `yield_lp()`. Saves the thread's context and lets the scheduler run the next ready thread.